Airspace Monitor

Project specification and architecture · Cloud & AI course

Team: Alek Hanušić, Dániel Csorba · 31 October 2026

Repository: https://github.com/airspace-monitor/airspace-live-tracker

1. Project specification

A web app that shows live air traffic on a world map, using data from the OpenSky Network. We collect the data ourselves, train models on it, and add an AI assistant on top. It is a non-commercial, educational project.

Features

Feature	What the user gets
Live map	Aircraft on a dark world map, coloured by altitude, moving smoothly between updates
Aircraft details	A selected aircraft's trail and its likely destinations
Unusual traffic	Hexagon map cells with unusually high or low traffic for that time of the week
Event feed	Emergencies, aborted landings, unusual traffic and gaps in data collection
Arrival board	Predicted arrival times, as a range, for flights to Vienna, Bratislava and Budapest
AI briefing	A short summary of the airspace, regularly updated and written from database facts
AI chat	A plain-English question, answered from our database, with the query shown
Dashboards	Data collection health, data quality and model results

Non-functional properties

Property	Target
Reliability	Data is saved locally before upload, so an outage only delays it. Gaps are recorded and shown. Under 5% of collection slots missed per week once collection runs 24/7
Freshness	New positions every 90 seconds, which is what OpenSky's daily API quota allows
Performance	The map stays smooth with 10,000+ aircraft on a normal laptop
Model quality	Arrival predictions beat a simple baseline. The chat answers at least 80% of a fixed test question set correctly
Security	Nothing is open to the internet. Access goes through a private network, each service has its own limited database role, and AI queries are read-only
Cost	As close to free as possible: the server is on a free tier, and paid AI usage has a hard spending limit
Maintainability	Services run in Docker Compose, database changes are versioned migrations, and code changes go through tests and pull requests
Data use	Non-commercial use with OpenSky attribution shown. OpenSky's terms are cleared with OpenSky before any public launch

2. What we use and where it runs

A computer at home collects the data and uploads it to one cloud server. The app runs privately for now. Both team members reach the server over Tailscale with their own accounts.

Running today

Part	Runs on	Built with
Data collector	Home PC for now, switched off overnight. A Raspberry Pi running 24/7 takes over when it arrives	Python
Private network	Collector, server and our laptops	Tailscale
Cloud server	Oracle Cloud free tier, Frankfurt: ARM, 2 CPUs, 12 GB RAM, 200 GB disk	Ubuntu, Docker Compose
Database	Server	PostgreSQL with TimescaleDB and PostGIS
Ingest service	Server	Python, FastAPI

Planned

Part	Runs on	Built with
Scheduled jobs, model training	Server	Dagster, LightGBM, MLflow
Live processing, web API	Server	Python, FastAPI
Web app	User's browser	React, MapLibre, deck.gl
Dashboards	Server	Grafana
AI briefing and chat	Called from the server	LLM API
Archives and backups	Off-site storage	Cloudflare R2
Automated tests and deployment	GitHub (deployed by hand for now)	GitHub Actions

Data sources: OpenSky live positions and daily flight lists, METAR airport weather reports, public reference data on airports, aircraft and airlines, and a historical flight dataset for typical routes.

Collected so far: since 29 October the pipeline has stored about 13.5 million aircraft positions from 1,245 snapshots and about 177,000 flight records, with no rows rejected. A snapshot holds about 10,800 aircraft on average.

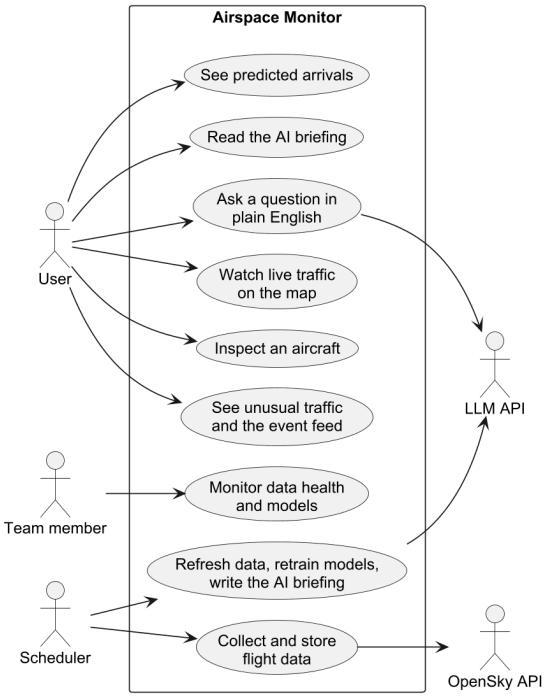
3. Architecturally significant use-cases

These five shape the architecture most. The other use-cases reuse the same structure.

Use-case	Why it matters	What it led to
Collect and store flight data	OpenSky blocks or throttles data-centre IPs, and home connections fail	A home collector that saves first, then pushes to the server; uploads that are safe to repeat
Watch live traffic	10,000+ aircraft must reach the browser and move smoothly	Database notifications, a live update signal to the browser, GPU rendering on the map
See unusual traffic	Needs weeks of history and summaries over many millions of rows	Collection from the start, a time-series database with automatic aggregates
See predicted arrivals	Models must behave the same in training and live use	Shared feature code, scheduled training, a model registry
Ask a question in plain English	AI-written SQL runs against our database	A read-only role, query checks, timeouts and usage limits

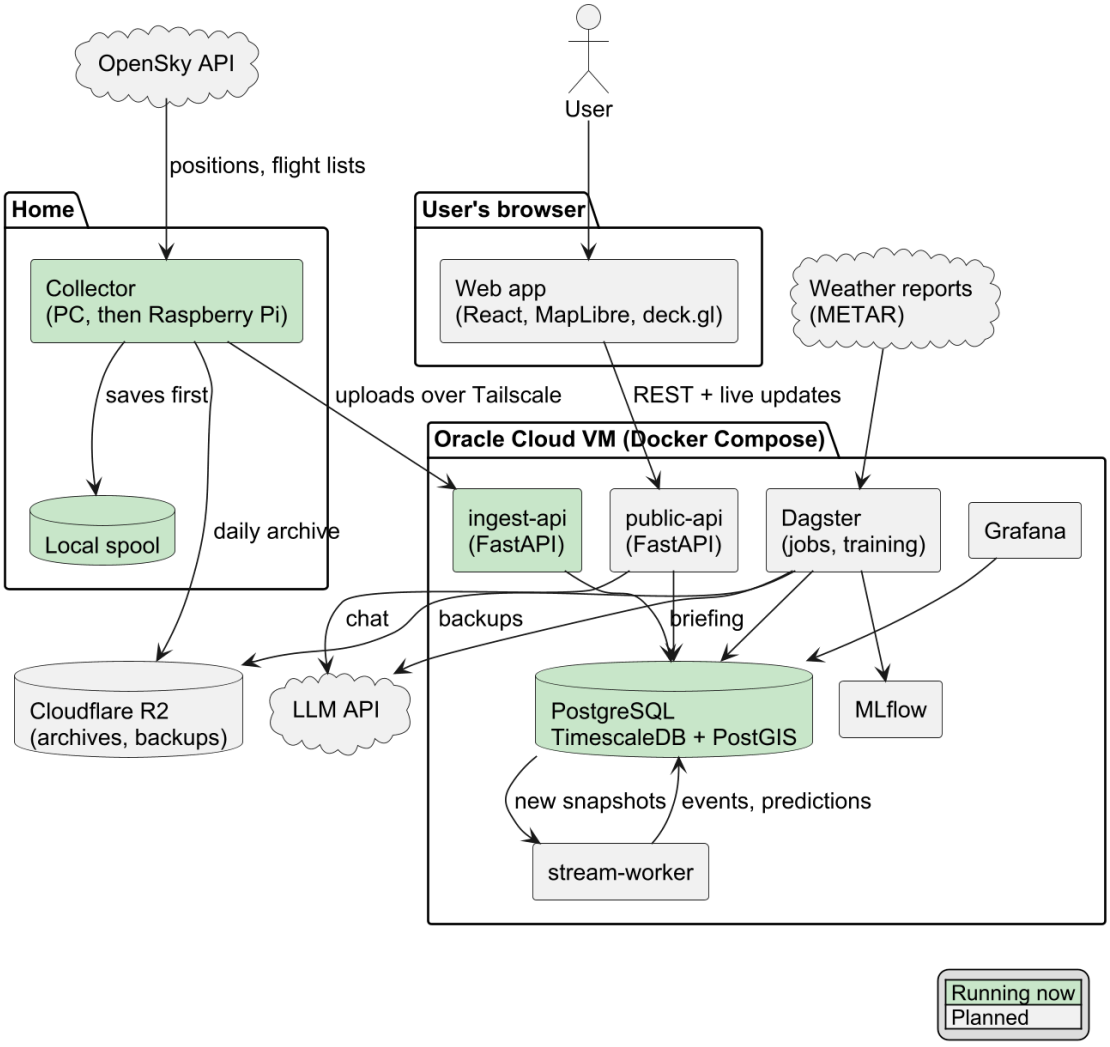
4. Architecture

4.1 Use-case model



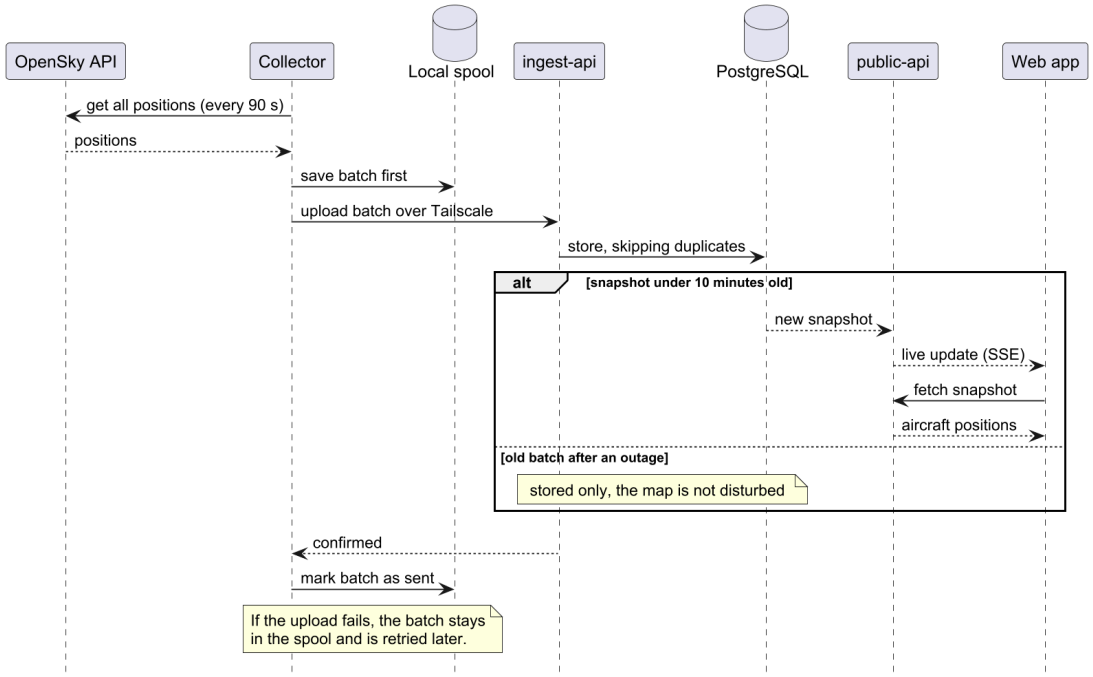
4.2 Component model

Green parts are running today. The rest are planned. Before any public launch, Cloudflare and a Caddy proxy would be added in front of the server.



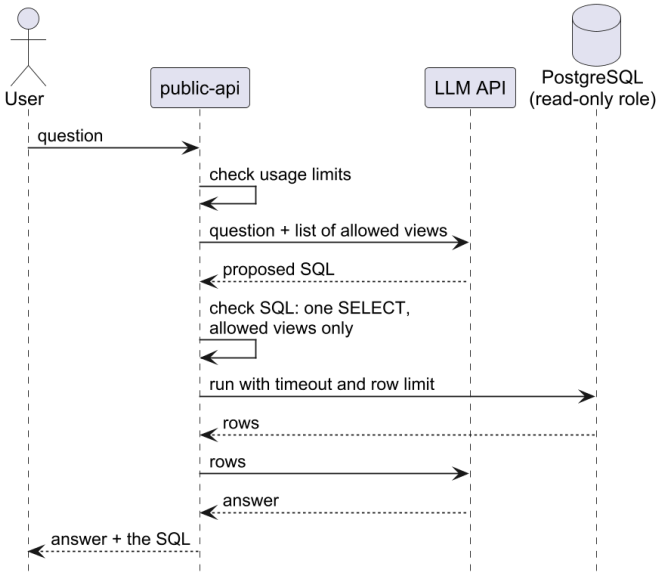
4.3 Sequence: from OpenSky to the live map

The batch is saved locally before upload. Old batches are stored without touching the live map.



4.4 Sequence: plain-English question

The AI never touches the database directly: the API checks every query before running it.



5. Architecture decisions

Decision	Why	Alternative considered
Collector at home pushes data to the cloud	OpenSky blocks or throttles data-centre IPs, so the server cannot fetch the data itself	Server polls OpenSky: likely to be blocked
Raspberry Pi as the collector	Quiet and low-power, so it can run 24/7. The PC is off at night, which leaves gaps	Leaving the PC on: noisy and uses more power
One collector, every 90 seconds	Simple, fits the API quota, and the map smooths movement between updates	More collectors for faster updates: more to run, little visible gain
Save locally first, uploads safe to repeat	Outages only delay data, and retries cannot create duplicates	Direct upload: simpler, but data is lost in an outage
PostgreSQL with TimescaleDB and PostGIS as the only database	One system for time-series, map and relational data. Plain SQL suits Grafana and the AI chat	Separate stores or streaming platforms: more to run, not needed at this scale
One free cloud VM with Docker Compose	Free, and simple enough for two people	Kubernetes: too much overhead. Paid VPS: kept as a fallback
Tailscale only while private	Nothing is exposed, and nothing extra is needed until a public launch	Public HTTPS now: needs a domain and more protection, with no users yet
Dagster for scheduled jobs	Retries, backfills and a view of run history	Cron: lighter, but no backfills or visibility
Database notifications and SSE for live updates	No extra infrastructure, and updates only flow one way	Kafka and WebSockets: not justified for one update every 90 seconds
Hexagon grid with a statistical baseline for unusual traffic	Explainable, and needs no labelled data	Machine-learning anomaly model: harder to evaluate, kept as a stretch goal
LightGBM for arrival times, route lookup for destinations	Strong on tabular data, cheap to train, and gives a time range. The lookup is simple to measure	Deep learning: more data and compute for little expected gain
AI chat writes SQL, with guardrails	Answers open questions, and showing the SQL keeps it transparent	Fixed set of queries: safer, but answers far fewer questions

6. Still open

Public launch. The app stays private for now, and going public is a possible later step. It would need a domain, Cloudflare and a Caddy proxy in front of the server, and clearing OpenSky's terms with OpenSky first.

Raspberry Pi. Until it takes over, collection stops overnight. These gaps are accepted and recorded, and the 5% target applies once it runs 24/7.

Model details. Hexagon size, thresholds and model features will be tuned on real data. Unusual-traffic detection needs at least three weeks of history.

Appendix: PlantUML source

Use-case model

```
@startuml use-cases
!pragma layout smetana
left to right direction
skinparam shadowing false
skinparam defaultFontName Arial
skinparam packageStyle rectangle

actor "User" as user
actor "Team member" as team
actor "Scheduler" as scheduler
actor "OpenSky API" as opensky
actor "LLM API" as llm

rectangle "Airspace Monitor" {
  usecase "Watch live traffic\non the map" as UC1
  usecase "Inspect an aircraft" as UC2
  usecase "See unusual traffic\nand the event feed" as UC3
  usecase "See predicted arrivals" as UC4
  usecase "Read the AI briefing" as UC5
  usecase "Ask a question in\nplain English" as UC6
  usecase "Monitor data health\nand models" as UC7
  usecase "Collect and store\nflight data" as UC8
  usecase "Refresh data, retrain models,\nwrite the AI briefing" as UC9
}

user --> UC1
user --> UC2
user --> UC3
user --> UC4
user --> UC5
user --> UC6
team --> UC7
scheduler --> UC8
scheduler --> UC9
UC8 --> opensky
UC6 --> llm
UC9 --> llm
@enduml
```

Component model

```
@startuml components
!pragma layout smetana
skinparam shadowing false
skinparam defaultFontName Arial
skinparam componentStyle rectangle
skinparam defaultFontSize 15

actor "User" as user
cloud "OpenSky API" as opensky
cloud "Weather reports\n(METAR)" as weather
cloud "LLM API" as llm
database "Cloudflare R2\n(archives, backups)" as r2

package "Home" {
  component "Collector\n(PC, then Raspberry Pi)" as collector #C8E6C9
  database "Local spool" as spool #C8E6C9
}

package "User's browser" {
  component "Web app\n(React, MapLibre, deck.gl)" as web
}

package "Oracle Cloud VM (Docker Compose)" {
  component "ingest-api\n(FastAPI)" as ingest #C8E6C9
  database "PostgreSQL\nTimescaleDB + PostGIS" as db #C8E6C9
  component "public-api\n(FastAPI)" as api
  component "stream-worker" as stream
  component "Dagster\n(jobs, training)" as dagster
  component "MLflow" as mlflow
  component "Grafana" as grafana
}

opensky --> collector : positions, flight lists
collector --> spool : saves first
collector --> ingest : uploads over Tailscale
collector --> r2 : daily archive
ingest --> db
db --> stream : new snapshots
stream --> db : events, predictions
user --> web
web --> api : REST + live updates
api --> db
api --> llm : chat
grafana --> db
dagster --> db
dagster --> mlflow
dagster --> llm : briefing
dagster --> r2 : backups
weather --> dagster

legend right
  |<#C8E6C9> Running now |
  |<#F1F1F1> Planned |
endlegend
@enduml
```

Sequence: OpenSky to live map

```
@startuml seq-collect-to-map
skinparam shadowing false
hide footbox
skinparam defaultFontName Arial

participant "OpenSky API" as O
participant "Collector" as C
database "Local spool" as S
participant "ingest-api" as I
database "PostgreSQL" as D
participant "public-api" as A
participant "Web app" as W

C -> O : get all positions (every 90 s)
O --> C : positions
C -> S : save batch first
C -> I : upload batch over Tailscale
I -> D : store, skipping duplicates
alt snapshot under 10 minutes old
  D --> A : new snapshot
  A --> W : live update (SSE)
  W -> A : fetch snapshot
  A --> W : aircraft positions
else old batch after an outage
  note over I, D : stored only, the map is not disturbed
end
I --> C : confirmed
C -> S : mark batch as sent
note over C, S
  If the upload fails, the batch stays
  in the spool and is retried later.
end note
@enduml
```

Sequence: plain-English question

```
@startuml seq-chat
skinparam shadowing false
hide footbox
skinparam defaultFontName Arial

actor "User" as U
participant "public-api" as A
participant "LLM API" as C
database "PostgreSQL\n(read-only role)" as D

U -> A : question
A -> A : check usage limits
A -> C : question + list of allowed views
C --> A : proposed SQL
A -> A : check SQL: one SELECT,\nallowed views only
A -> D : run with timeout and row limit
D --> A : rows
A -> C : rows
C --> A : answer
A --> U : answer + the SQL
@enduml
```